

Chapter 4* Rasterization Practice

• 核心

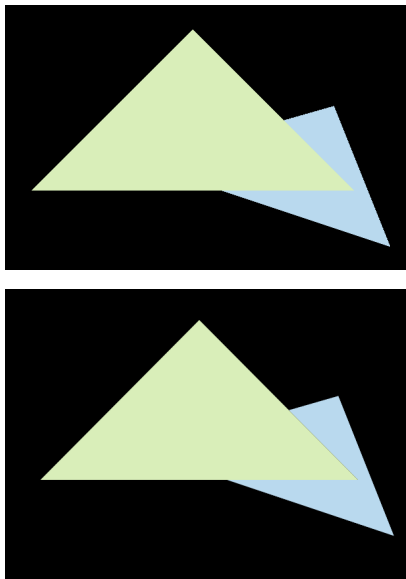
• 理解并实现三角形栅格化方法

- 步骤一：定义primitives。定义视口大小（像素）、眼睛位置、三角形（所有点的坐标、indices表示哪些点构成一个三角形、rgb每个点的颜色）
- 步骤二：构建mvp。构建mvp变换矩阵，即model transformation, view transformation以及projection transformation

注意这部分需要和原坐标独立开来，因为每次改变视角都需要重新对原坐标进行变换

- 步骤三：hv变换。实现homogeneous division（又称perspective division），以及viewport transformation（将图像放缩至视口的大小）
- 步骤四：rasterization。实现光栅化，即确定每一个像素的着色。这部分包含两步操作，第一步是判断像素点是否需要着色，第二步是确定像素点的着色

- 步骤4.1：判断像素点是否需要着色。首先叉积判断像素点是否在三角形内，其次用中心坐标插值插出当前点的深度并与Z-buffer中的值比较
- 步骤4.2：确定像素点的着色。可以使用某一点的颜色为整个三角形着色，也可以利用插值的方法实现渐变着色
- 附加：可以采用Super-sampling方法处理走样，对比图如下所示



• 其它

• Eigen

- 了解了Eigen库中点积（`a.dot(b)`）以及叉积（`a.cross(b)`）的用法
- 了解了Eigen库中`head<n>()`函数的用法：对于Eigen::Vector4f/Vector3f类型的变量x，`x.head<n>()`代表提取前n个元素

- C/C++

- 了解了C中enum变量类型的用法

实例

```
#include <stdio.h>

enum DAY
{
    MON=1, TUE, WED, THU, FRI, SAT, SUN
};

int main()
{
    enum DAY day;
    day = WED;
    printf("%d",day);
    return 0;
}
```

以上实例输出结果为:

3

- 了解了C中lambda表达式的使用 `[&/=](params){}`。其中 `[]` 为捕获列表，即按引用/值捕获上下文的变量，在函数体中直接使用
- 了解了main函数的形参 `void main(int argc,char **argv)`
`argc` 代表参数个数， `argv` 代表参数， `argv[0]` 默认为程序地址